

```

1.class Solution {
    public boolean halvesAreAlike(String s) {
        String vowels = "aeiouAEIOU";
        int mid = s.length() / 2;
        int countA = 0, countB = 0;

        for (int i = 0; i < mid; i++) {
            if (vowels.indexOf(s.charAt(i)) != -1) countA++;
            if (vowels.indexOf(s.charAt(i + mid)) != -1) countB++;
        }

        return countA == countB;
    }
}

```

Input

s =
"book"

Output

true

Expected

true

2. import sys

from collections import Counter

def solve():

input_data = sys.stdin.read().split()

if not input_data:

return

T = int(input_data[0])

results = []

```

for i in range(1, T + 1):
    s = input_data[i]
    n = len(s)

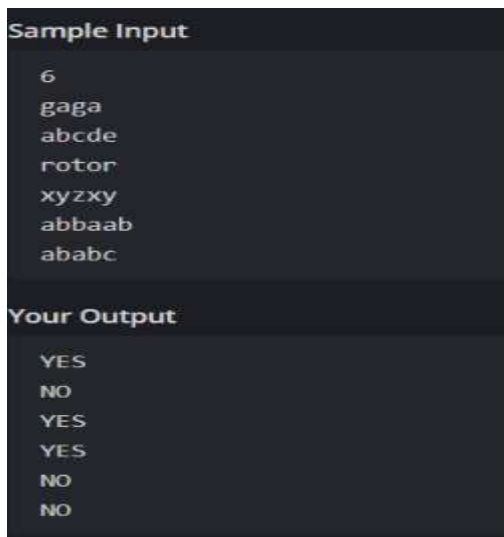
    half_len = n // 2
    first_half = s[:half_len]
    second_half = s[n - half_len:]

    if Counter(first_half) == Counter(second_half):
        results.append("YES")
    else:
        results.append("NO")

print("\n".join(results))

if __name__ == "__main__":
    solve()

```



```

Sample Input
6
gaga
abcde
rotor
xyzxy
abbaab
ababc

Your Output
YES
NO
YES
YES
NO
NO

```

```

3. import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;

```

```
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

class Result {

    public static List<Integer> compareTriplets(List<Integer> a, List<Integer> b) {

        int aliceScore = 0;
        int bobScore = 0;

        for (int i = 0; i < 3; i++) {
            if (a.get(i) > b.get(i)) {
                aliceScore++;
            } else if (a.get(i) < b.get(i)) {
                bobScore++;
            }
        }

        List<Integer> result = new ArrayList<>();
        result.add(aliceScore);
        result.add(bobScore);
        return result;
    }
}

public class Solution {

    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
```

```
BufferedWriter bufferedWriter = new BufferedWriter(new  
FileWriter(System.getenv("OUTPUT_PATH")));
```

```
List<Integer> a = Stream.of(bufferedReader.readLine().replaceAll("\\s+$", "").split(" "))  
    .map(Integer::parseInt)  
    .collect(toList());
```

```
List<Integer> b = Stream.of(bufferedReader.readLine().replaceAll("\\s+$", "").split(" "))  
    .map(Integer::parseInt)  
    .collect(toList());
```

```
List<Integer> result = Result.compareTriplets(a, b);
```

```
bufferedWriter.write(  
    result.stream()  
        .map(Object::toString)  
        .collect(joining(" "))  
        + "\n"  
);
```

```
bufferedReader.close();
```

```
bufferedWriter.close();
```

```
}
```

```
}
```



```
4. import java.util.HashSet;
```

```

class Solution {
    public boolean containsDuplicate(int[] nums) {

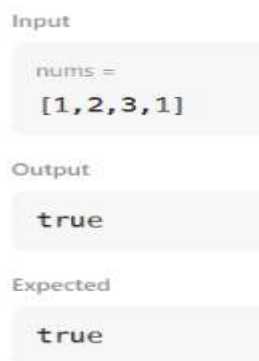
        HashSet<Integer> set = new HashSet<>();

        for (int num : nums) {
            if (set.contains(num)) {
                return true;
            }

            set.add(num);
        }

        return false;
    }
}

```



```

5.import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;

```

```

import java.util.stream.*;

import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

class Result {

    public static String timeConversion(String s) {

        int hour = Integer.parseInt(s.substring(0, 2));

        String ampm = s.substring(8, 10);

        if (ampm.equals("AM")) {
            if (hour == 12) {
                hour = 0;
            }
        } else {
            if (hour != 12) {
                hour += 12;
            }
        }

        return String.format("%02d", hour) + s.substring(2, 8);
    }
}

public class Solution {

    public static void main(String[] args) throws IOException {

        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));

        BufferedWriter bufferedWriter = new BufferedWriter(new
        FileWriter(System.getenv("OUTPUT_PATH")));

        String s = bufferedReader.readLine();

```

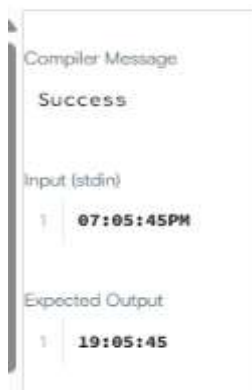
```

String result = Result.timeConversion(s);

bufferedWriter.write(result);
bufferedWriter.newLine();

bufferedReader.close();
bufferedWriter.close();
}
}

```



```

6. class Solution{
    public void moveZeroes(int[] nums){
        int j=0;
        for (int i=0; i<nums.length; i++){
            if(nums[i]!=0){
                nums[j]=nums[i];
                j++;
            }
        }
        while (j<nums.length){
            nums[j]=0;
            j++;
        }
    }
}

```

```
}  
}
```

Input

```
nums =  
[0,1,0,3,12]
```

Output

```
[1,3,12,0,0]
```

Expected

```
[1,3,12,0,0]
```

```
7. import java.io.*;  
import java.math.*;  
import java.security.*;  
import java.text.*;  
import java.util.*;  
import java.util.concurrent.*;  
import java.util.function.*;  
import java.util.regex.*;  
import java.util.stream.*;  
import static java.util.stream.Collectors.joining;  
import static java.util.stream.Collectors.toList;  
  
class Result {  
  
    public static int diagonalDifference(List<List<Integer>> arr) {  
  
        int n = arr.size();  
  
        int leftSum = 0;
```

```

int rightSum = 0;

for (int i = 0; i < n; i++) {
    leftSum += arr.get(i).get(i);
    rightSum += arr.get(i).get(n - 1 - i);
}

return Math.abs(leftSum - rightSum);
}
}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));

        BufferedWriter bufferedWriter = new BufferedWriter(new
        FileWriter(System.getenv("OUTPUT_PATH")));

        int n = Integer.parseInt(bufferedReader.readLine().trim());

        List<List<Integer>> arr = new ArrayList<>();

        IntStream.range(0, n).forEach(i -> {
            try {
                arr.add(
                    Stream.of(bufferedReader.readLine().replaceAll("\\s+$", "").split(" "))
                        .map(Integer::parseInt)
                        .collect(toList())
                );
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        });
    }
}

```

```

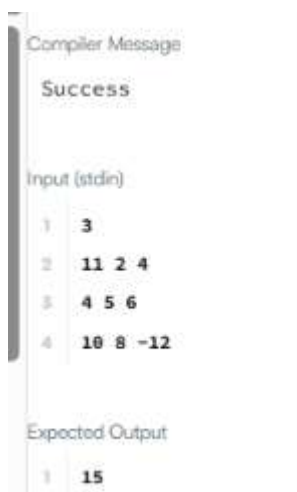
});

int result = Result.diagonalDifference(arr);

bufferedWriter.write(String.valueOf(result));
bufferedWriter.newLine();

bufferedReader.close();
bufferedWriter.close();
}
}

```



```

8. class Solution {
    public int[][] transpose(int[][] matrix) {

        int rows = matrix.length;
        int cols = matrix[0].length;

        int[][] result = new int[cols][rows];

        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {

```

```

        result[j][i] = matrix[i][j];
    }
}

return result;
}
}

```

Input

```

matrix =
[[1,2,3],[4,5,6],[7,8,9]]

```

Output

```

[[1,4,7],[2,5,8],[3,6,9]]

```

Expected

```

[[1,4,7],[2,5,8],[3,6,9]]

```

```

9. class Solution {
    public int[][] matrixBlockSum(int[][] mat, int k) {

        int m = mat.length;
        int n = mat[0].length;

        int[][] answer = new int[m][n];

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {

                int sum = 0;

```

```

        int startRow = Math.max(0, i - k);
        int endRow = Math.min(m - 1, i + k);

        int startCol = Math.max(0, j - k);
        int endCol = Math.min(n - 1, j + k);

        for (int r = startRow; r <= endRow; r++) {
            for (int c = startCol; c <= endCol; c++) {
                sum += mat[r][c];
            }
        }

        answer[i][j] = sum;
    }
}

return answer;
}
}

```

Input

```

mat =
[[1,2,3],[4,5,6],[7,8,9]]

```

```

k =
1

```

Output

```

[[12,21,16],[27,45,33],[24,39,28]]

```

Expected

```

[[12,21,16],[27,45,33],[24,39,28]]

```

```

10import java.io.*;

```

```
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

class Result {

    public static void matrixRotation(List<List<Integer>> matrix, int r) {

        int m = matrix.size();
        int n = matrix.get(0).size();

        int layers = Math.min(m, n) / 2;

        for (int layer = 0; layer < layers; layer++) {

            List<Integer> values = new ArrayList<>();

            // Top row
            for (int j = layer; j < n - layer; j++) {
                values.add(matrix.get(layer).get(j));
            }

            // Right column
            for (int i = layer + 1; i < m - layer; i++) {
```

```
        values.add(matrix.get(i).get(n - layer - 1));  
    }
```

```
// Bottom row
```

```
for (int j = n - layer - 2; j >= layer; j--) {  
    values.add(matrix.get(m - layer - 1).get(j));  
}
```

```
// Left column
```

```
for (int i = m - layer - 2; i > layer; i--) {  
    values.add(matrix.get(i).get(layer));  
}
```

```
// Rotate anti-clockwise
```

```
int size = values.size();
```

```
int shift = r % size;
```

```
List<Integer> rotated = new ArrayList<>();
```

```
for (int i = shift; i < size; i++) {  
    rotated.add(values.get(i));  
}
```

```
for (int i = 0; i < shift; i++) {  
    rotated.add(values.get(i));  
}
```

```
// Put rotated values back
```

```
int index = 0;
```

```
// Top row
```

```

        for (int j = layer; j < n - layer; j++) {
            matrix.get(layer).set(j, rotated.get(index++));
        }

        // Right column
        for (int i = layer + 1; i < m - layer; i++) {
            matrix.get(i).set(n - layer - 1, rotated.get(index++));
        }

        // Bottom row
        for (int j = n - layer - 2; j >= layer; j--) {
            matrix.get(m - layer - 1).set(j, rotated.get(index++));
        }

        // Left column
        for (int i = m - layer - 2; i > layer; i--) {
            matrix.get(i).set(layer, rotated.get(index++));
        }
    }

    // Print result
    for (List<Integer> row : matrix) {
        System.out.println(
            row.stream()
                .map(String::valueOf)
                .collect(Collectors.joining(" "))
        );
    }
}

public class Solution {

```

```

public static void main(String[] args) throws IOException {

    BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));

    String[] firstMultipleInput = bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");

    int m = Integer.parseInt(firstMultipleInput[0]);

    int n = Integer.parseInt(firstMultipleInput[1]);

    int r = Integer.parseInt(firstMultipleInput[2]);

    List<List<Integer>> matrix = new ArrayList<>();

    IntStream.range(0, m).forEach(i -> {
        try {
            matrix.add(
                Stream.of(bufferedReader.readLine().replaceAll("\\s+$", "").split(" "))
                    .map(Integer::parseInt)
                    .collect(toList())
            );
        } catch (IOException ex) {
            throw new RuntimeException(ex);
        }
    });

    Result.matrixRotation(matrix, r);

    bufferedReader.close();
}
}

```

Input (stdin)

```
1 4 4 1
2 1 2 3 4
3 5 6 7 8
4 9 10 11 12
5 13 14 15 16
```

Expected Output

```
1 2 3 4 8
2 1 7 11 12
3 5 6 10 16
4 9 13 14 15
```